A Flying Breadboard

A Product Development Story

How a US$13 display board, a barometric breakout and four days of AI-assisted integration replaced weeks of guessing about how a variometer behaves in real air.

R. Malik · Vario project engineering programme

Sources: vario-usb-workbook (commit 53907ab, corrected flight benchmark) · ESP32-S3-Display-DataLogger repository

Companion article: From Flight Data to Filter Decisions

Date: 2026-11-10 · Website publication handoff

1 · The product, and the question we could not answer on the bench

The product under development is a small pressure-sensor variometer for paragliding: a device that reads a barometric pressure sensor, estimates vertical speed in real time, and tells the pilot — acoustically — when the wing is climbing in lift or sinking out. The engineering requirement sounds simple: **report vertical speed as early as possible, without false alarms and without chattering at the pilot when the air is neutral.**

By the time this story begins, the programme had already produced a working bench infrastructure: an ESP32-S3 test rig with two barometers and an IMU, a causal-replay filter benchmark, frozen configurations for a bank of candidate estimators, an audio policy, and a defect register documenting how earlier conclusions were corrected when the evidence changed. The bench told us a lot. It could not tell us the one thing that mattered next: **how does the system behave in real air?**

All of our motion evidence was proxy evidence. Elevator rides imitate slow, clean altitude changes; hand shakes imitate vibration; static sessions imitate standing on launch. None of them reproduce the actual operating environment — a paraglider climbing in a turbulent thermal at +2 m/s, slicing through sink, flying in air that changes character every few seconds. Filter tuning, audio-threshold selection and sensor validation carried an open risk of being optimized for the bench rather than for the sky. Closing that gap required **real flight measurements** — raw pressure from the actual sensor, recorded on an actual flight, at the full sample rate the algorithms run at.

2 · Why we needed a temporary data logger

The bench rig was never designed to fly: it streams over USB to a host computer at 921 600 baud, is powered from the USB port, and lives on a desk. The options were weighed directly:

Table 1. *Alternatives considered for obtaining real flight pressure data.*

Option	Verdict
Fly the bench rig with a laptop	Rejected — no practical way to host a laptop in flight; USB link and power both fail.
Buy a consumer GPS / barometer logger	Rejected — low-rate position/altitude logs; no synchronized 40 Hz raw pressure stream, no estimator state, no integrity guarantees aligned with our processing chain.
Phone barometer apps	Rejected for the same reason: no raw-stream access, no integrity guarantees, no alignment with our processing chain.
Build a small temporary logger	**Selected.** Same BMP581 sensor already validated on the bench, same estimator bank, battery-powered, logging to SD card, cheap and fast.

The logger was explicitly a **supporting development tool, not a product**. Its job was to reduce one specific uncertainty: what does the pressure stream actually look like in flight, and what do our frozen algorithms do with it? Everything about its design follows from treating it as disposable tooling.

3 · Hardware selection: the cheapest thing that removes the uncertainty

The selection logic was deliberately inverted from a product design. Instead of asking “what should the final hardware be?”, we asked “what is the cheapest, fastest assembly that records the same pressure stream our product senses, at the same rate, with verifiable integrity?”

Table 2. *Hardware bill of the temporary flight logger.*

Item	Choice	Rationale
MCU + display	Waveshare ESP32-S3-LCD-1.9 dev board, ~ US$13	Integrated ESP32-S3 (240 MHz, 16 MB flash, 8 MB octal PSRAM), LCD for field status, USB for bench streaming — nothing else to integrate at board level.
Barometer	BMP581 breakout (I²C 0x47)	The exact sensor already validated in the bench programme (40 Hz cadence read, on-chip compensation Pa = raw24/64). Flight data directly comparable to every earlier capture.
IMU	Onboard QMI8658C; ST LSM6DSV16X breakout kept, not fitted	Motion context if needed, without blocking the barometer path.
Audio	Passive piezo buzzer (GPIO16, LEDC PWM)	In-flight acoustic behaviour identical in kind to the product's virtual audio.
Storage	microSD via onboard SDMMC slot	Self-contained recording independent of any host link.
Assembly	Breadboard and jumper wiring	Fast, fully re-wirable, inspectable. The flight would last minutes; the tooling did not need to survive a season.

Why no custom PCB? A PCB is an investment in a design freeze. At this stage we did not know what the flight would teach us — that is the point of the exercise. Layout, fabrication and rework would add weeks and cost without reducing any of the uncertainty the flight was meant to resolve.

Why no enclosure? The logger needed to survive one experiment on a cushioned harness, not a production lifetime. A 3D-printed shell is a product decision; the tooling version needs zip ties and foam.

Why no more powerful processor? Measurement, not computation, was the bottleneck. Earlier phases had already measured on-target CPU cost for the full estimator bank at well under 3% of the real-time tick. The ESP32-S3 was vastly more compute than the logging task required; a bigger processor would have added nothing but board cost and power draw.

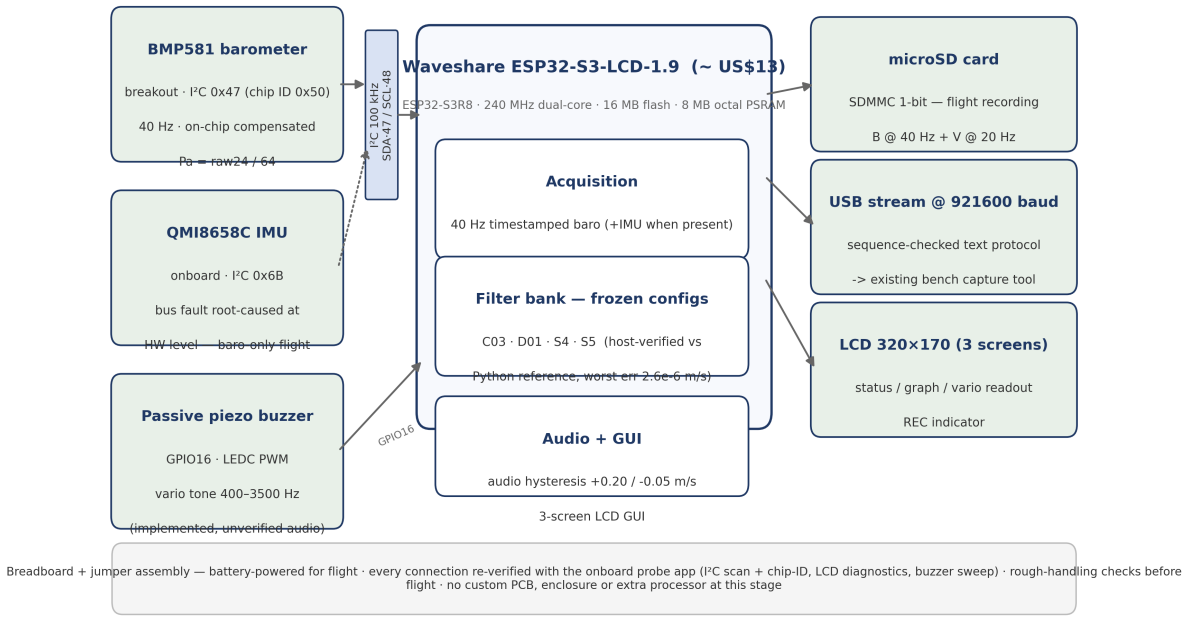


Figure 1. *Block diagram of the breadboard flight logger: sensor set, shared I²C bus, ESP32-S3 processing chain and recording outputs, drawn from the hardware table of the DataLogger repository.*

4 · Assembly, and trusting nothing you have not poked

The system was assembled on a breadboard: board, BMP581 breakout and buzzer on a shared I²C bus, SD card in the onboard slot. Breadboard construction under vibration is exactly as fragile as it sounds, so **verifiability was designed in rather than assumed**:

• A **probe-mode firmware** build was kept as the first-class diagnostic: on demand it runs an I²C scan with WHO_AM_I / CHIP_ID disambiguation (the BMP581 answers chip ID 0x50), draws a labelled-corner LCD diagnostic frame, and sweeps the buzzer.

• **Connection testing was repeated, not assumed** — after assembly changes and before flight, the scan/ID path re-verified every device on the bus.

• **Rough-handling checks** stood in for flight vibration: the assembled unit was shaken and tapped while the stream ran, watching for dropouts, I²C wedges or SD write stalls — the failure modes that would silently corrupt a flight recording.

• The honest-status convention from the bench programme carried over: every feature is labelled *verified on hardware*, *implemented but unverified*, or *deferred*. The buzzer tone, for example, shipped as “implemented, not acoustically verified” until it was heard working — a small habit that prevents bench confidence from masquerading as flight readiness.

5 · Four days of AI-assisted integration — built on verified foundations

The full logger firmware — 40 Hz timestamped acquisition, the four-estimator filter bank, three-screen GUI, audio state machine, USB streaming protocol and SD recording path — came together in **approximately four days of AI-assisted development**. The speed is real, but the sentence needs its second half: **it was fast because it was almost entirely recombination of previously verified engineering work**, and every reused piece had been earned earlier:

• The **filter bank** (C03, D01, S4, S5) was not rewritten. The frozen, benchmark-validated Python references were ported to C and **host-verified against the Python originals to a worst error of 2.6e-6 m/s** before any flight.

• The **BMP581 driver** came from the workbook's validated bring-up (0x47, cadence read at 40 Hz, on-chip compensation).

• The **USB streaming protocol** (sequence counters, handshake, meta frames) was lifted from the bench capture toolchain, so the flight recorder spoke exactly the language the existing analysis pipeline parsed.

• The earlier programme's discipline — frozen configurations, defect register, replay verification — meant the new tool inherited a test culture, not just code.

AI compressed the implementation phase dramatically. It did not compress the verification phase, and it did not make engineering judgment optional. Two examples from the same four days:

The LCD refused to render correctly through several rounds of “obvious” fixes; the actual root causes were a missing COLMOD (pixel-format) command in the panel init sequence, an active-low backlight polarity inversion, and an LSB-first font bit order. Each was found by instrumented diagnosis, not by accepting the first plausible patch.

The onboard IMU initially appeared dead. Rather than accept that or blame the driver, the investigation walked the whole chain — driver, bus speed, bit-banged access, reset sequences — before attributing the fault at the hardware level. The logger flew barometer-only, and the decision was *evidence-based*, not convenience-based.

The pattern worth keeping: **AI makes the first draft cheap; engineering verification makes it true.** The faster implementation becomes, the more the value of an experienced engineer shifts to deciding what to build, what to trust, and what must be proven before the result is believed.

6 · The flight

The assembled breadboard logger — display board, barometer breakout, buzzer, microSD card — flew on a real paragliding flight. In-flight photographs of the unit under the harness are held as externally supplied assets (see the asset manifest) and will accompany the website article.

The recording it brought back exceeded what the bench could ever provide:

• **761.65 seconds** of device time — launch, sustained thermalling between 900 and 990 m, and landing rollout.

• **30,467 barometric samples at 40.0 Hz** with zero sequence gaps, duplicates, malformed lines or timestamp discontinuities.

• A coherent flight profile: climb-out from ~927 m at launch, repeated thermal climbs and transitions, flare, and a quiet landing rollout.

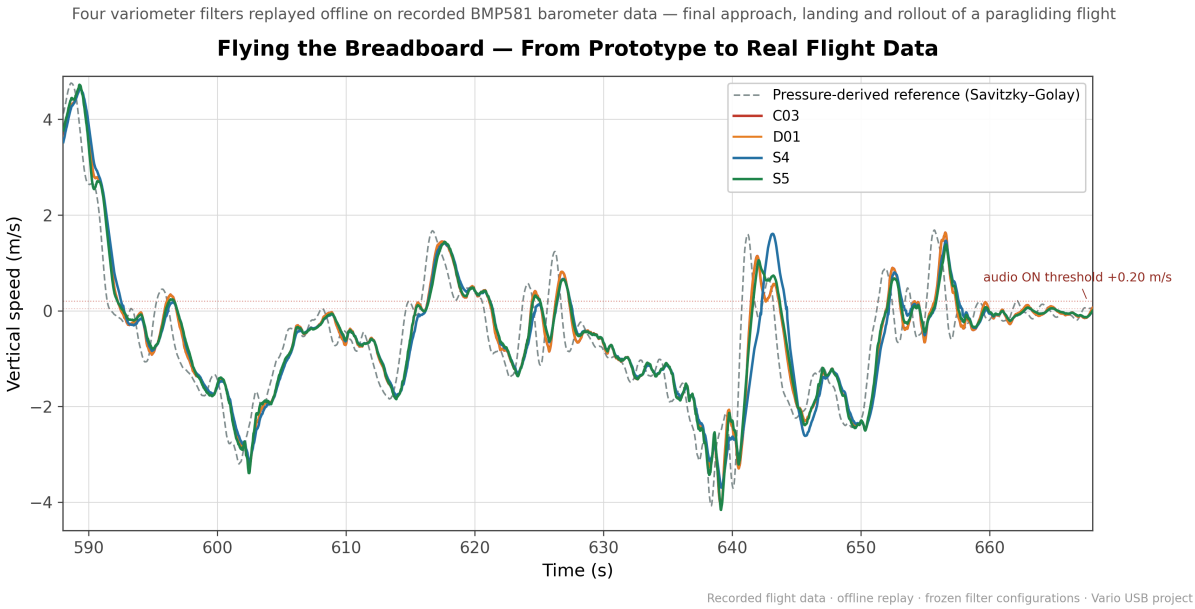


Figure 2. *Final approach, landing and rollout of the flight (t = 588–668 s): four frozen variometer filters replayed offline on the recorded BMP581 stream, against the pressure-derived reference and the +0.20 m/s audio threshold. The steep sink burst, flare and quiet rollout visible here are the data no bench test could produce. Verified plot, reports/flight/linkedin_flying_the_breadboard.png.*

One afternoon of real air produced more decision-relevant information than the preceding weeks of proxy testing: what the pressure stream actually looks like at thermal onsets, how much lag the filters really exhibit against the pressure-derived reference (about **0.6–0.7 s**, after a correction to the analysis that had initially reported ~2.1 s — see the companion article), how the audio policy behaves in turbulent, near-threshold air, and which responsiveness-versus-smoothness trade-off actually matters to a pilot.

7 · What the breadboard bought us

The logger answered the flight-data question for the cost of a US$13 board, a breakout sensor, a handful of jumpers and about four days of work. Concretely, it:

• **Eliminated the largest open uncertainty** in the programme — whether bench-derived filter behaviour survives contact with real air. It does, and now we know quantitatively where and by how much the bench was right.

• **Upgraded the filter discussion from rankings to trade-offs** — on flight data the candidates differentiate on quietness, chatter and transient fidelity, not on correctness. That is a design conversation, not a bug hunt.

• **Validated the sensor and recording chain end-to-end** — the BMP581 at 40 Hz with zero integrity defects over 12.7 flight-minutes, and the SD recording path under real vibration.

• **Told us what to build next** — IMU + GPS in the flight recorder, multiple estimators logged simultaneously, and a quiet-soaring flight to firm up the quietness metrics. Requirements discovered from evidence, not assumed.

8 · Lessons

• **Engineering judgment decides what deserves to be built.** The custom PCB, the enclosure and the bigger processor were all reasonable products — and all wrong tools for this question. The discipline is in matching the investment to the uncertainty: spend on what reduces risk, defer what does not.

• **Rapid feedback beats speculative perfection.** Four days of tooling plus one flight replaced an unbound amount of bench extrapolation. A rough answer to the right question is worth more than a precise answer to the wrong one.

• **AI reduces implementation time; it does not eliminate verification.** It changed what the engineer's day looks like — more deciding, diagnosing and checking; less typing — but the record of what is verified, and the responsibility for it, stays human.

• **A breadboard can fly.** Not elegantly, not forever — but long enough to turn a guess into a measurement. That is often all a development tool needs to do.

Sources and evidence trail

• ESP32-S3-Display-DataLogger repository (github.com/raghavmalik89): hardware table, pinout, probe-mode diagnostics, filter-port verification (worst error 2.6e-6 m/s), module status matrix.

• vario-usb-workbook repository (github.com/raghavmalik89): bench rig, filter benchmark phases, BMP581 bring-up, USB protocol, on-target CPU measurements, flight benchmark at commit 53907ab.

• Companion article: *From Flight Data to Filter Decisions* — the corrected flight benchmark and all quantitative filter findings referenced above.

• In-flight photographs: externally supplied assets (owner to provide; see asset manifest).